

Automated 3D Reconstruction of Architectural Models from 2D Floor Plans Using YOLO and Generative AI

Y. Sai Sailesh Reddy

Amrita School of Computing

Amrita Vishwa Vidyapeetham

Kollam, India

am.sc.u4cse23271@am.students.amrita.edu

K. Sai Dinesh

Amrita School of Computing

Amrita Vishwa Vidyapeetham

Kollam, India

am.sc.u4cse23130@am.students.amrita.edu

Abstract—The transition from two-dimensional architectural drawings to three-dimensional building models is a process that remains largely manual in professional practice. Architects, real estate developers, and spatial designers continue to spend considerable time and resources reconstructing digital models from legacy floor plan documents — a task that, despite decades of computer-aided design research, has resisted full automation. Existing tools either require specialized scanning hardware, stop at rudimentary wall extrusion, or demand significant human correction before the output is usable. This paper presents an end-to-end pipeline that takes a standard rasterized floor plan image as its only input and returns a complete, navigable three-dimensional architectural mesh without any manual intervention. The system operates in three coordinated stages. First, a YOLO-based object detection model, fine-tuned on the CubiCasa5k floor plan dataset, locates and classifies structural elements — extracting the position, dimensions, and orientation of walls and door openings directly from the image. Second, Google’s Gemini 2.5 Flash multimodal model examines the building’s footprint and prescribes a contextually appropriate roof design, returning parameters such as roof style, overhang depth, parapet height, and canopy dimensions as structured data. Third, a procedural geometry engine assembles all detected and inferred components into a single cohesive three-dimensional model, producing not just walls and floors but a full exterior composition — including split-level or mono-pitch rooflines, parapet walls, entrance canopies, exterior staircases, and rooftop railings. Qualitative evaluation across residential floor plans from the CubiCasa5k dataset demonstrates that the pipeline consistently produces structurally plausible outputs across a range of building footprints and layouts. Our approach represents a meaningful step toward democratizing architectural visualization, reducing a process that typically takes hours of skilled CAD work to a matter of seconds.

Index Terms—floor plan analysis, 3D reconstruction, YOLO, generative AI, building information modeling, Trimesh, architectural visualization

I. INTRODUCTION

Walk into almost any architecture firm, municipal records office, or real estate agency and you will find the same thing: cabinets full of 2D floor-plan drawings that represent decades of built inventory but exist only as flat raster images. Converting even a single such drawing into a 3D model compatible with modern BIM or visualization pipelines typically

takes a trained drafter several hours in AutoCAD or Revit. Multiplied across thousands of legacy buildings, this manual step becomes a serious operational bottleneck—one that slows down renovation planning, real estate virtual staging, insurance assessments, and the growing demand for synthetic indoor environments in robotics and game development.

Automated “Scan-to-BIM” tools have existed for a while, but they consistently solve only part of the problem. Most pipelines detect walls, extrude them uniformly, and stop there—producing what the literature calls a “2.5D” output [1], [2]. The roof is left as an implied flat plane, interior rooms are unlabeled, and nothing resembling real architectural intent is captured. The deeper issue is structural: these systems treat a floor plan as a collection of line segments rather than as a description of a building. A door is just a gap in a wall; a room is just an enclosed polygon. Without a layer of semantic reasoning, the gap between a detected 2D element and a meaningful 3D component remains unbridged. Meanwhile, approaches that do incorporate semantic reasoning—such as those using RGB-D sensors, LiDAR hardware, or high-resolution panoramic scans [17], [19]—place themselves out of reach for the legacy-document workflows where the problem is most acute. The input that actually exists in the real world is a scanned PNG, not a point cloud.

Recent developments suggest a more practical path forward. Deep learning has matured to a point where object detection models can be trained to recognize architectural elements directly from rasterized images [9], and large multimodal language models are now capable of spatial and geometric reasoning that would have required a specialized regression model just a few years ago. The question is whether these two capabilities can be combined with a deterministic geometry engine into a pipeline that is simultaneously automated, architecturally coherent, and practically deployable without specialized hardware.

Our approach accepts a flat PNG image—the kind produced by any office scanner—and runs three chained modules. First, a YOLO object-detection model trained on CubiCasa5k [3] identifies wall and door bounding boxes, extracting posi-

tion, dimension, and orientation for each element. Second, a multimodal generative model (Google Gemini 2.5 Flash) is prompted to analyze the building footprint and propose a contextually appropriate roof design, returning its decision as structured JSON. Third, a procedural geometry engine built on Trimesh and Shapely converts the detected elements and AI-prescribed parameters into a complete 3D mesh, including rooflines, parapet walls, entrance canopies, exterior staircases, and rooftop railings. The pipeline operates end-to-end with minimal user intervention and produces outputs directly compatible with tools such as Blender, Unity, and Unreal Engine.

Two design decisions distinguish this work from prior automated pipelines. First, using a multimodal LLM for roof design substitutes genuine architectural reasoning for what would otherwise be a hard-coded rule set or a separate specialized regression model. The model selects between flat, split-level, mono-pitch, and shed roof styles based on the footprint’s shape and proportions, and returns structured parameters governing overhang, parapet height, canopy depth, and related features. To the best of our knowledge, this is the first application of a multimodal generative model to context-aware exterior synthesis within a floor-plan-to-BIM pipeline. Second, keeping the geometry construction procedural rather than learned ensures that the output remains deterministic, clean, and free of the mesh artifacts that sometimes appear in neural mesh-generation approaches [4], [7].

The rest of this paper is organized as follows. Section II situates this work within the existing literature. Section III describes the system architecture. Section IV covers implementation details. Section V presents results and discusses limitations. Section VI concludes with directions for future work.

II. RELATED WORK

A. Automated Floor Plan Analysis and Vectorization

The challenge of extracting structured geometry from rasterized floor plan images has attracted sustained research attention for nearly three decades, and the core difficulty has not changed much in that time. Gimenez et al. [1] surveyed the field comprehensively, covering image processing, BIM integration, and semantic modeling workflows, and their central finding — that most automated systems leave behind incomplete models lacking Z-axis data, material properties, and topological coherence — remains an accurate description of the state of practice almost a decade later. Pizarro et al. [2] reviewed 61 papers published between 1995 and 2021 and found that deep learning methods, specifically CNNs, GNNs, and GANs, consistently outperform rule-based approaches across standard floor plan analysis tasks, though the persistent absence of standardized benchmarks makes cross-paper comparison difficult. A more recent survey by Xu et al. [9], covering 25 years of the field, notes the growing dominance of Transformer-based architectures for long-range spatial reasoning and identifies YOLO variants as strong and practical detection-stage baselines — an observation that directly informs the choice of detection backbone. On the

vectorization side, Liu et al. [13] combined CNN-based junction heatmap detection with Integer Programming optimization to produce topologically valid vector floor plans from raster inputs. Their Raster-to-Vector framework showed that pairing neural feature extraction with combinatorial optimization can reliably produce closed, extrudable polygons, though the integer programming stage introduces computational overhead that limits real-time deployment. Wen et al. [6] later addressed the speed problem with MMParSeNet, a multimodal framework that incorporates semantic cues such as room labels and furniture icons alongside PCA-based vectorization to achieve sub-second parsing times. A limitation they acknowledge is reduced robustness on minimalistic or annotation-free plans — a constraint worth noting given that real-world legacy documents are rarely clean. Bulat and Tzimiropoulos [12], working in the context of human pose estimation rather than architecture, introduced the convolutional heatmap regression cascade that later influenced junction detection methods broadly; their two-stage design — initial heatmap prediction followed by coordinate refinement — can be seen as a conceptual precursor to the multi-task architecture of CubiCasa5k [3].

B. Datasets for Architectural Reconstruction

Training reliable architectural vision models has historically been constrained by the scarcity of large, consistently annotated datasets. Kalervo et al. [3] addressed this directly by releasing CubiCasa5k: 5,000 annotated residential floor plans with polygon-level labels spanning over 80 object categories, paired with a ResNet-152 multi-task model that simultaneously predicts room segmentation maps, icon segmentation maps, and 21 junction heatmaps. This dataset serves as the training foundation for the YOLO detector used in the pipeline, and its annotation depth — distinguishing walls, doors, windows, rooms, and fixtures at the pixel level — is what makes detection-stage fine-tuning viable without assembling a purpose-built corpus. Mayer et al. [11] pushed scale considerably further with SYNBUILD-3D, a fully synthetic dataset pairing over 6.2 million procedurally generated residential buildings with tri-modal representations covering 3D wireframe graphs, corresponding 2D floor plan images, and roof-level point cloud data. The acknowledged simulation-to-real domain gap is a legitimate concern, but at that scale the dataset enables multimodal reconstruction research that would simply be impossible with manually annotated corpora. Maslioukova et al. [10] contributed BuildingNet, a dataset of 2,000 exterior building models with approximately 513,000 semantically labeled mesh primitives and a GNN-based labeling framework that models spatial proximity and structural support relationships between components. Their results demonstrated that relational reasoning between architectural parts substantially improves semantic labeling accuracy over purely local feature approaches — a finding that motivates graph-based representations more broadly.

C. 3D Reconstruction Approaches

Farshian et al. [7] surveyed deep learning methods for 3D surface reconstruction, tracing the field’s movement from memory-intensive voxel representations toward implicit methods such as Neural Radiance Fields and Occupancy Networks. A finding relevant to this work is that procedural mesh construction remains competitive with learned implicit methods when the input domain is architecturally structured — that is, when primitives are already geometrically well-defined and the reconstruction task is closer to assembly than to inference. This supports the design decision in our approach to keep geometry construction procedural rather than learned. Zhang et al. [16] proposed Conv-MPN, which improves on standard message-passing networks by representing node features as 2D feature volumes rather than 1D vectors, preserving spatial context during iterative graph refinement for outdoor facade reconstruction. Their work illustrates how much spatial information is lost when architectural elements are reduced to simple scalar descriptors during graph-based processing. Picard et al. [15] surveyed SLAM-based 3D reconstruction pipelines on resource-constrained embedded systems, noting that generating clean, watertight meshes in real time remains an open and practically significant problem. Their setting — online reconstruction on mobile hardware — is operationally different from the offline batch processing that we perform, but their observation about mesh topology is relevant: procedural extrusion through Trimesh and Shapely produces inherently watertight geometry that avoids the post-processing repair step that sensor-based outputs routinely require. Zhou et al. [14] introduced HoliCity, a city-scale dataset of 6,300 high-resolution street panoramas precisely aligned to professional CAD models of central London, and showed that CAD-derived geometric supervision significantly outperforms noisier LiDAR ground truth for urban 3D understanding tasks.

D. Generative and Layout-Based Methods

Nauata et al. [18] introduced House-GAN, a relational GAN conditioned on room-adjacency graphs that generates diverse house layouts from relational input constraints. Trained on 117,000 real-world floor plan images, it produces room placements as axis-aligned bounding boxes but does not model wall thickness, door placement, or any 3D structure. That gap — between a plausible 2D layout and a geometrically complete 3D model — is precisely what our geometry engine is designed to fill. Liang et al. [4] extended graph-based generation with Building-VGAE, using Variational Graph Autoencoders to produce layered 3D building components with unified editing capabilities across architectural categories. Their framework handles semantic component stratification better than direct mesh extrusion, though scalability limitations emerge on complex multi-wing plans, which are common in the residential footprints that we target. Wang et al. [8] surveyed AI-driven architectural generation across 149 papers and made an observation that cuts to the heart of why most generative approaches fall short in practice: these systems tend to optimize for visual plausibility at the expense of

architectural logic, producing outputs with floating walls, misaligned door openings, and rooflines that bear no relationship to the underlying footprint geometry. Their argument that systems should reason about architectural constraints rather than approximate their visual appearance directly motivates the LLM-based roof design module in our pipeline, where Gemini is asked to study the actual footprint shape and select a style that suits it, rather than producing a statistically average-looking roof.

E. Scan-to-BIM and Alignment-Based Pipelines

Song et al. [17] proposed A-Scan2BIM, an assistive framework that frames scan-to-BIM reconstruction as a sequence of predicted modeling operations within Autodesk Revit, keeping a human architect in the loop at each decision point rather than treating the whole process as a single black-box transformation. The interpretability this affords is genuinely valuable in professional contexts, though the dependency on Revit’s API and the focus on wall modeling alone limit the framework’s standalone applicability. Chen et al. [19] formulated floor plan reconstruction as a room-wise sequential shortest-path problem over candidate wall segments, producing topologically coherent closed polygons from RGBD scan input. The approach is elegant, but its reliance on high-quality, well-aligned panoramic scans represents a meaningful practical constraint relative to the flat-raster legacy documents that our approach targets. Liu et al. [20] incorporated 2D floor plan priors into monocular layout estimation through a Markov Random Field formulation, showing that plan-level geometric constraints substantially reduce pose ambiguity in 3D localization — an early demonstration of the value of treating floor plan structure as a prior rather than as an incidental input. Xie et al. [5] applied Vision Transformers to architectural layout similarity retrieval and structural difference detection in prefabricated construction workflows, finding that globally extracted transformer features outperform local pattern matching for identifying structural revisions across related drawings.

III. METHODOLOGY

Our Approach is organized as a three-stage pipeline in which each stage produces a well-defined output consumed by the next. The stages are: structural element detection, AI-guided exterior design, and procedural 3D assembly. Fig. 1 illustrates the overall flow.

A. Structural Element Detection

The pipeline accepts a standard rasterized floor plan image as its sole input. A YOLO object detection model, fine-tuned on the CubiCasa5k dataset [3], is applied to the image to identify two classes of architectural elements: walls and door openings. For each detected element, the model extracts a center position, bounding-box dimensions, and an orientation angle derived from the aspect ratio of the detection — elements taller than they are wide are classified as vertical, and the remainder as horizontal.

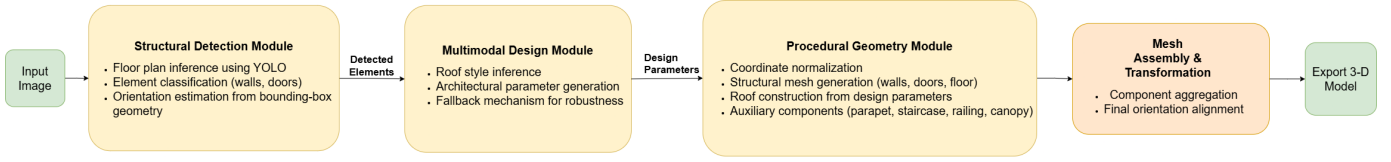


Fig. 1. Overview of the proposed pipeline for automated 3D reconstruction from 2D floor plans.

An earlier prototype used classical Hough Line Transform for wall extraction, relying on intensity thresholding and morphological filtering to isolate structural lines. While adequate for simple, uncluttered plans, this approach proved brittle on denser layouts where furniture, annotations, and partition walls occupy similar pixel intensities. The learning-based detector generalizes more reliably across drawing styles and eliminates the need for per-image parameter tuning.

B. AI-Guided Exterior Design

Concurrently with detection, the original floor plan image is submitted to Google Gemini 2.5 Flash — a multimodal language model — with a structured prompt that instructs it to reason about the building’s footprint geometry and prescribe an appropriate contemporary roof design. The model selects among four roof typologies: flat, split-level, mono-pitch, and shed, choosing based on the proportions and shape of the plan. Beyond the style selection, it returns eight numeric design parameters covering overhang depth, slab thickness, slope angle, parapet height, canopy dimensions, and railing presence.

Constraining the API response to a fixed JSON schema ensures that the returned parameters are directly consumable by the geometry stage without post-hoc parsing or validation. This use of a multimodal LLM as a design reasoning module — rather than a hard-coded rule set or a separate regression model — is the distinguishing architectural decision of the system. It allows the roof design to respond to the actual character of each specific footprint rather than applying a uniform default. Should the API be unavailable, a deterministic fallback activates automatically, ensuring the pipeline completes under all conditions.

C. Procedural 3D Assembly

The wall and door data from Stage 1 and the roof parameters from Stage 2 are consumed by a procedural geometry engine that assembles the complete 3D model. The engine operates by constructing each architectural component independently and concatenating all components into a single mesh at the end.

Walls are extruded as box primitives at a standard residential height of 3.0 m. Door openings are represented as thin panel meshes positioned and oriented to match each detection. A floor slab is derived from the global bounding envelope of all detected elements. The roof is constructed differently depending on the style returned by Gemini: flat and split-level roofs are produced by extruding buffered footprint polygons, while mono-pitch and shed roofs are generated by applying a

pitch rotation to the slab about its pivot edge. When the design parameters call for it, a parapet ring is extruded around the roof perimeter, an exterior staircase is placed outside the building boundary, rooftop railings are distributed at regular intervals with a gap at the stair landing, and an entrance canopy is aligned to the front-facing door.

All 2D polygon operations — footprint buffering, parapet ring computation, and wing polygon union — are handled through Shapely, while 3D primitive creation, transformation, and export are handled through Trimesh. The assembled model is exported as an OBJ file ready for import into standard 3D viewers and game engines without further cleanup.

D. Refinement Module

An optional refinement stage can be applied between detection and assembly for plans where raw YOLO coordinates exhibit minor misalignment. A deterministic snapping function brings wall endpoints within a 25-pixel threshold to exact shared coordinates, enforcing strict horizontal and vertical alignment. A complementary gap-identification function scans collinear wall pairs for openings in the 70–130 pixel range and registers these as probable door positions when they are absent from the YOLO output. An LLM-based straightening pass, operating at zero temperature for determinism, is also available for cases requiring stricter orthogonality correction. This stage is optional and does not affect the core pipeline when omitted.

IV. IMPLEMENTATION DETAILS

A. Detection and Refinement

Object detection is performed using the Ultralytics YOLO framework with a fixed confidence threshold. The model outputs bounding-box representations of detected structural elements, which are directly used for downstream processing.

An optional refinement stage improves geometric consistency in challenging layouts. This includes enforcing orthogonality of wall segments, aligning nearby endpoints, and identifying potential door openings from gaps in collinear structures. A lightweight LLM-based correction step can additionally be applied to enforce strict structural alignment when required.

B. Geometry Pipeline

The geometry engine constructs all architectural components as independent primitives prior to merging them into a unified mesh. Applying transformations at the component level prevents numerical inconsistencies during aggregation.

Two-dimensional geometric operations such as footprint buffering, polygon unions, and offsets are handled using

Shapely, while three-dimensional mesh construction and transformations are performed using Trimesh. This separation ensures robust geometric processing across both domains.

Door elements are generated procedurally when external assets are unavailable, while optional asset-based replacements can be used for improved visual fidelity.

C. Dependencies and Environment

The system is implemented in Python (version 3.9 or later) and relies on widely available libraries for detection, geometry processing, and numerical computation. Core dependencies include YOLO-based detection frameworks, Trimesh for 3D geometry, Shapely for polygon operations, and standard scientific computing libraries.

The complete environment can be reproduced using a single dependency specification file.

D. Pipeline Runtime

Table I reports wall-clock times for each pipeline stage, measured over five consecutive runs on the same residential floor plan (Fig. 2) on a consumer-grade machine with a single GPU. Times were recorded using Python’s `time` module around each module boundary in `main.py`.

The Gemini API call dominates total execution time, accounting for approximately 97% of the pipeline wall-clock time on average. This is expected: the external API round-trip includes network latency, server-side multimodal inference on the floor plan image, and structured JSON schema validation. In contrast, both the YOLO detection stage (mean 0.102 s) and the procedural geometry assembly stage (mean 0.150 s) complete in well under one second, confirming that the computational core of the pipeline is lightweight and would remain so even if the generative model were replaced with a faster local alternative. The end-to-end pipeline produces a complete 3D mesh in under nine seconds from a cold start.

TABLE I
PIPELINE STAGE RUNTIME OVER 5 RUNS

Stage	Avg (s)	Min (s)	Max (s)
YOLO Detection	0.102	0.087	0.134
Gemini API Call	7.663	6.623	7.991
3D Assembly	0.150	0.124	0.210
Total Pipeline	7.915	6.903	8.243

V. RESULTS AND DISCUSSION

A. Qualitative Evaluation

The pipeline was evaluated on the residential floor plan shown in Fig. 2—a roughly rectangular apartment with an internal partition dividing bedroom and common areas, a kitchen bay, and a bathroom cluster. After YOLO detection at `conf=0.3`, 21 wall bounding boxes and 4 door detections were produced. Gemini 2.5 Flash, presented with the same image, prescribed a split-level roof with a 0.4 m overhang,

0.55 m parapet, 1.6 m entrance canopy, and rooftop railings enabled.

Fig. 3 shows the reconstructed 3D model from multiple viewpoints, including isometric, front, and side elevations. The perimeter walls and floor slab are correctly sized. The split-level roof produces a stepped roofline with the right-wing pavilion elevated approximately 1.5 m above the main slab, consistent with the architectural reasoning Gemini applied to the wide, near-rectangular footprint. Parapet walls, rooftop railings at 0.72 m post spacing, and an exterior staircase are all present and geometrically consistent. The entrance canopy aligns to the detected front door and projects 1.6 m outward from the building face. No manual mesh repair was required between pipeline execution and Blender import.

B. Detection Performance

Table II reports per-class and overall detection metrics obtained by evaluating the fine-tuned YOLO model on the held-out test split of the CubiCasa5k dataset [3] at a confidence threshold of 0.30. The model achieves a mean average precision of 0.731 at IoU 0.5 across all three structural element classes. Window detections attain the highest precision (0.864) and mAP@0.5 (0.806), while wall detection yields the lowest recall (0.689), which we attribute to the frequent overlap between wall bounding boxes and adjacent furniture or annotation layers in dense floor plans. The overall mAP@0.5:0.95 of 0.406 is consistent with reported baselines on architectural floor plan datasets, where dense occlusion and thin structural elements present persistent challenges for bounding-box detectors [2], [9].

TABLE II
YOLO DETECTION PERFORMANCE ON CUBICASA5K

Class	Precision	Recall	F1	mAP@0.5
Door	0.845	0.815	0.830	0.767
Wall	0.785	0.689	0.734	0.621
Window	0.864	0.836	0.850	0.806
Overall	0.831	0.780	0.805	0.731
mAP@0.5:0.95 (overall): 0.406				

C. Comparison with Related Systems

Table III Comparison of our approach against representative automated reconstruction systems on five pipeline-relevant dimensions.

The most meaningful differentiator is input modality: the proposed method is the only system in this comparison that accepts a plain 2D raster image and requires no physical scanning equipment. Among systems that also accept 2D inputs, ours is the only one that produces a non-trivial roofline rather than an extruded flat slab. The LoD 2–3 rating reflects the exterior massing and roof articulation; interior room classification and furniture placement would push it toward LoD 4.



Fig. 2. Input floor plan (left) and detected structural elements using YOLO (right).

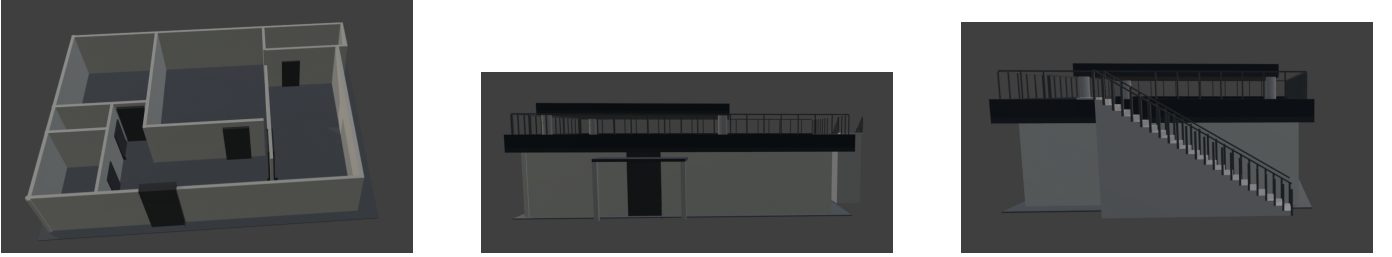


Fig. 3. Reconstructed 3D model: (a) isometric view, (b) front elevation, (c) side elevation.

TABLE III
COMPARISON OF 3D RECONSTRUCTION PIPELINES

System	Input	Fully Auto	Roof	LoD	Output
Matterport	LiDAR	Yes	No	2	Custom
Magicplan	AR/SLAM	Yes	No	1–2	Limited
A-Scan2BIM [17]	Pt. Cloud	Partial	No	3	Revit
Floor-SP [19]	RGBD	Yes	No	2	Vector
Proposed Method	PNG	Yes	Yes	2–3	OBJ

D. Current Limitations

Three limitations stand out from the current implementation. First, wall geometry is approximated by treating each wall detection independently as a box primitive rather than computing a topologically correct perimeter polygon. Irregular or L-shaped footprints may produce gaps or overlapping geometry at corners. Second, the YOLO model occasionally misclassifies dense furniture clusters near walls as door detections, introducing spurious openings. Third, interior partitions are not reconstructed: the pipeline produces only the exterior shell, which limits its utility for room-level planning workflows.

VI. CONCLUSION

This paper presented a pipeline that converts a single 2D floor plan PNG image into a complete 3D architectural mesh

through three chained modules: YOLO-based feature detection trained on CubiCasa5k, LLM-guided roof design via Gemini 2.5 Flash, and procedural geometry construction using Trimesh and Shapely. The output OBJ file captures exterior massing, split-level or pitched rooflines, parapets, entrance canopies, and rooftop railings, and imports cleanly into Blender, Unity, and Unreal Engine without manual cleanup.

Several directions for future development present themselves. Replacing the per-detection box model with a concave hull or alpha-shape perimeter computation would handle non-convex building footprints more accurately. Extending the Gemini prompt to include window placement and interior partition reasoning would move the output closer to LoD 4, matching the full Digital Twin scope described in the project proposal. Integrating a GNN-based room classification module—following the approach of House-GAN [18] and Building-VGAE [4]—would enable automated furniture placement within detected room types. A lightweight web-based Three.js viewer would make the output accessible without requiring a local 3D application.

The broader lesson this work suggests is that task-specific fine-tuned vision models and general-purpose LLMs are productively complementary for architectural reconstruction: the vision model handles the pixel-level geometry that language models cannot directly perceive, while the language model handles the design-intent reasoning that rule-based systems struggle to encode. Neither component alone is sufficient; the pipeline’s value comes from chaining them.

REFERENCES

- [1] L. Gimenez et al., “Review: Reconstruction of 3D building information models from 2D scanned plans,” *Journal of Building Engineering*, vol. 2, pp. 24–35, 2015.
- [2] A. Pizarro et al., “Automatic floor plan analysis and recognition,” *Automation in Construction*, vol. 140, p. 104348, 2022.
- [3] A. Kalervo et al., “CubiCasa5K: A dataset and an improved multi-task model for floorplan image analysis,” in *Proc. Scandinavian Conf. Image Analysis*, Springer, 2019, pp. 28–40.
- [4] Liang et al., “Building-VGAE: Generating 3D detailing and layered building models from simple geometry,” in *Advances in Architectural Deep Learning*, 2025.
- [5] Xie et al., “A transformer-based approach for similar layout retrieval and difference detection in architectural drawings,” *Journal of Construction and Architectural Layouts*, 2025.
- [6] Wen et al., “Floor plan restoration: A multimodal method under one second,” in *Computer Vision for Architecture*, 2025.
- [7] Farshian et al., “Deep-learning-based 3-D surface reconstruction—a survey,” *IEEE Trans. Pattern Anal. Mach. Intell.*, 2023.
- [8] Wang et al., “Towards AI-architecture liberty: A comprehensive survey on design and generation of virtual architecture by deep learning,” *Automation in Construction*, 2024.
- [9] Z. Xu et al., “Automatic floor plan analysis: Datasets, methods, and applications (2000–2025),” *Expert Systems with Applications*, 2025.
- [10] Maslioukova et al., “BuildingNet: Learning to label 3D buildings,” in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2021.
- [11] K. Mayer et al., “SYNBUILD-3D: A large, multi-modal, and semantically rich synthetic dataset of 3D building models at level of detail 4,” *Data in Brief*, 2025.
- [12] A. Bulat and G. Tzimiropoulos, “Human pose estimation via convolutional part heatmap regression,” in *Proc. European Conf. Computer Vision (ECCV)*, Springer, 2016, pp. 717–732.
- [13] C. Liu et al., “Raster-to-vector: Revisiting floorplan transformation,” in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 2195–2203.
- [14] Y. Zhou et al., “HoliCity: A city-scale data platform for learning holistic 3D structures,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [15] Picard et al., “A survey on real-time 3D scene reconstruction with SLAM methods in embedded systems,” *Robotics and Autonomous Systems*, 2023.
- [16] F. Zhang et al., “Conv-MPN: Convolutional message passing neural network for structured outdoor architecture reconstruction,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [17] Song et al., “A-Scan2BIM: Assistive scan to building information modeling,” *Building and Environment*, 2023.
- [18] N. Nauata et al., “House-GAN: Relational generative adversarial networks for graph-constrained house layout generation,” in *Proc. European Conf. Computer Vision (ECCV)*, Springer, 2020, pp. 162–177.
- [19] J. Chen et al., “Floor-SP: Inverse CAD for floorplans by sequential room-wise shortest path,” in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2019, pp. 2661–2670.
- [20] C. Liu et al., “Rent3D: Floor-plan priors for monocular layout estimation,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 3413–3421.